IES JUAN DE GARAY · 2.º DAW · PROYECTO INTERMODULAR II · TAREA 2

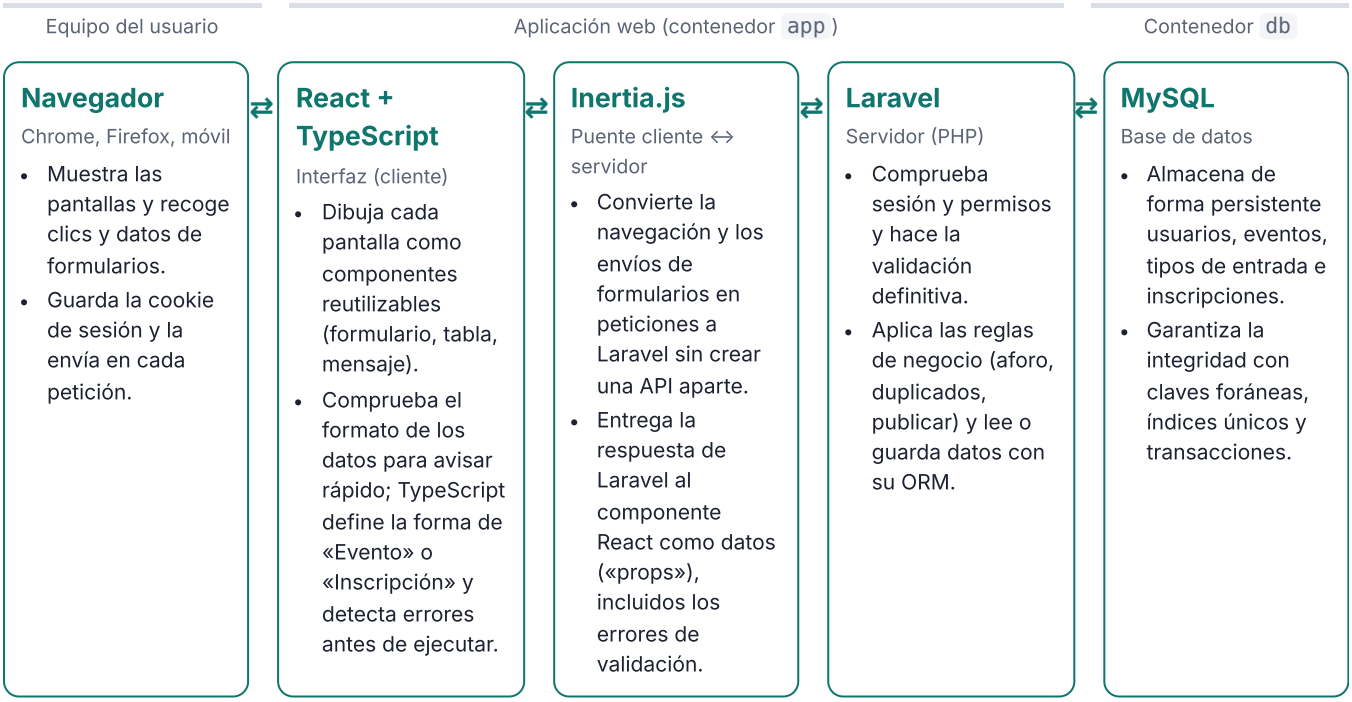
Arquitectura objetivo y tratamiento de datos

Laravel · React · TypeScript · Inertia.js · MySQL

Autor: Oleksandr Rudynskyy **Repositorio:** github.com/tokelad/Attendize **Fecha:** 28/10/2026

> Primera propuesta guiada. Describe la función general de cada parte; los nombres reales de rutas, controladores y clases se decidirán en las tareas de implementación.

1. Diagrama de la arquitectura

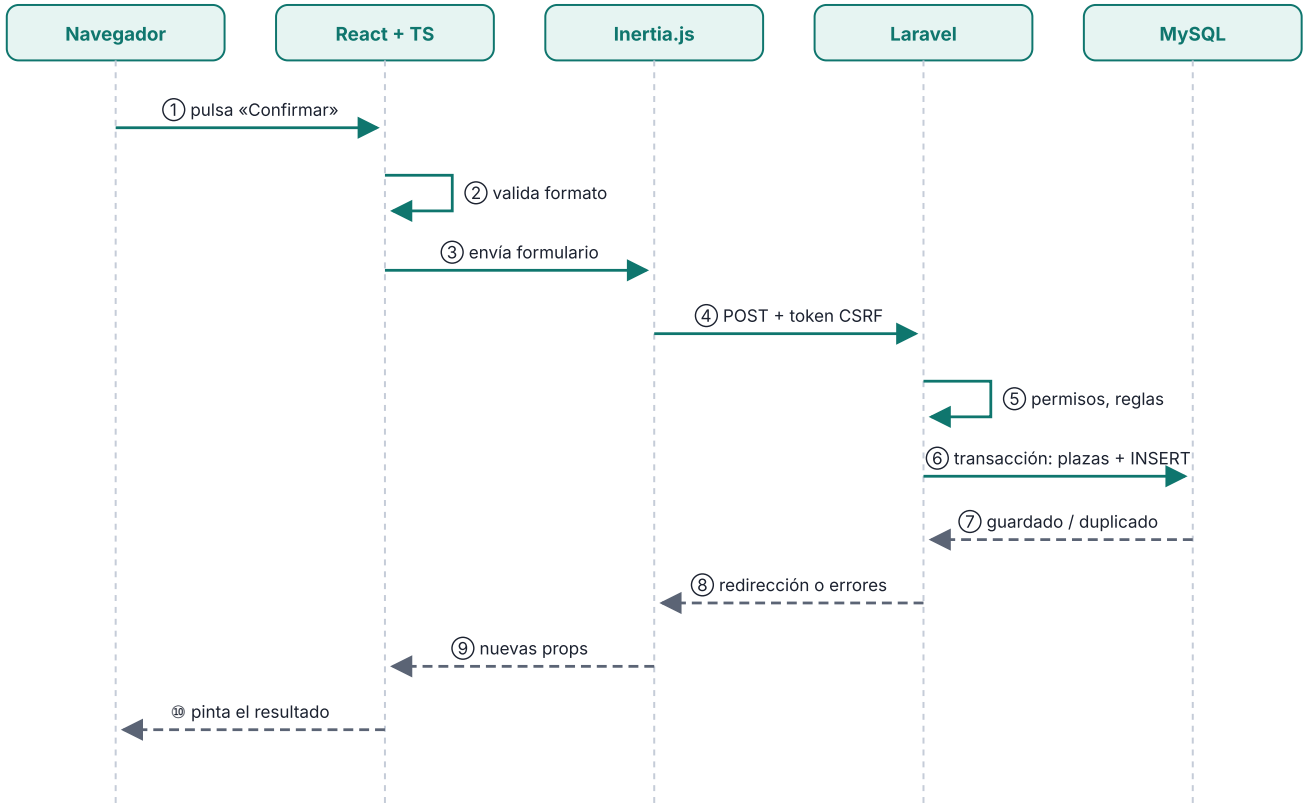


Servicios de apoyo heredados de la Tarea 1, no imprescindibles para el núcleo: cola con Redis y *worker* para enviar correos (RF-10) y MailDev para recibirlos en local.

Por qué estas tecnologías

Tecnología	Justificación para este proyecto
React	Las pantallas comparten piezas (campo con error, tabla que se convierte en tarjetas, estado vacío). Con componentes se escriben una vez y se reutilizan. Además, los formularios reaccionan al momento sin recargar la página.
TypeScript	Los datos que llegan del servidor tienen una forma fija (un evento siempre tiene título, fechas y aforo). Declararla evita errores como usar un campo que no existe, que en la versión actual (JavaScript sin tipos) solo se descubren al ejecutar.
Inertia.js	Permite usar React manteniendo las rutas, la sesión y la validación de Laravel, sin construir ni proteger una API REST separada. Es más sencillo para un proyecto individual.
Laravel	Es el framework de Attendize, así que se aprovecha lo aprendido en la Tarea 1. En una versión con soporte, aporta autenticación, validación, ORM, migraciones y colas.
MySQL	Es la base de datos actual. Mantenerla facilita migrar los datos existentes y permite garantizar en la propia base de datos que no haya inscripciones duplicadas.

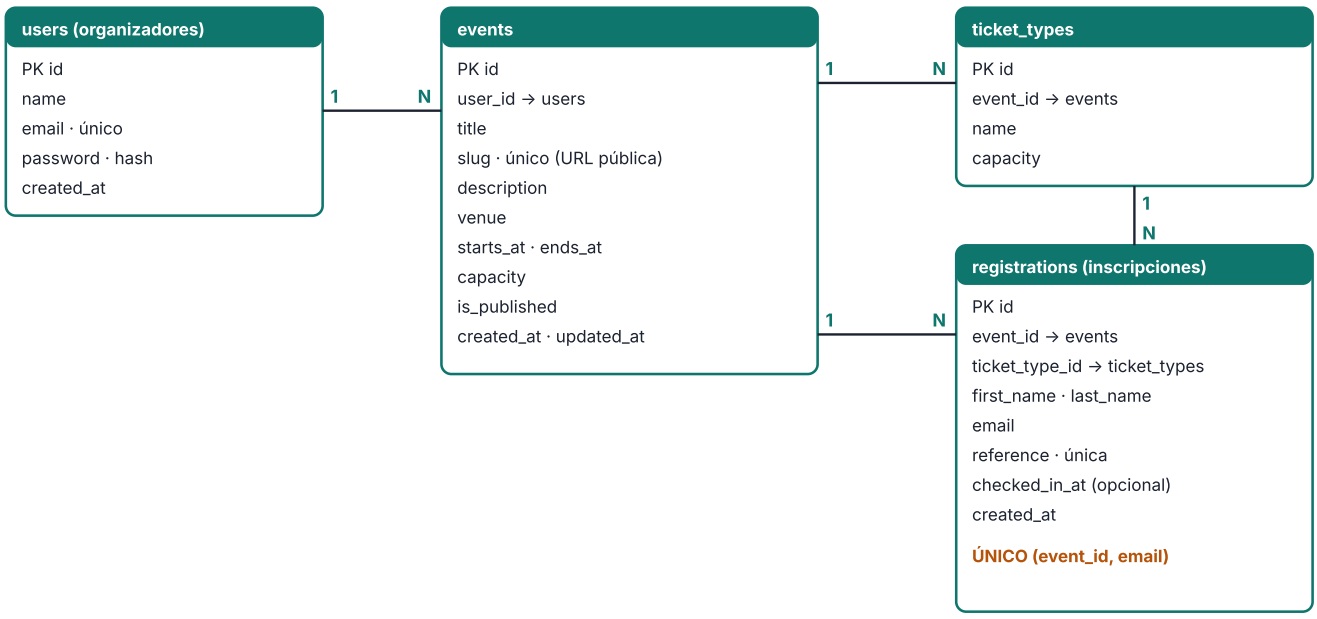
2. Recorrido de una petición: inscripción de un asistente (RF-06, RF-07)



1. **Acción del usuario.** En la página pública del evento, el asistente rellena nombre, apellidos y email, elige un tipo de entrada y pulsa «Confirmar inscripción».
2. **Comprobación en la interfaz.** React revisa el formato (email válido, campos obligatorios, casilla de datos). Si algo falla, avisa al instante sin enviar nada. Esta comprobación solo ayuda al usuario y no sustituye a la del servidor.
3. **Envío.** React entrega los datos a Inertia.js.
4. **Petición al servidor.** Inertia.js envía una petición HTTP a Laravel con el token CSRF que protege frente a envíos falsificados.
5. **Procesamiento en Laravel.** Comprueba que el evento está publicado, vuelve a validar todos los campos, normaliza el email (minúsculas, sin espacios) y prepara la inscripción con una referencia única.
6. **Acceso a MySQL.** Dentro de una transacción, Laravel consulta las plazas libres del tipo de entrada y, si quedan, guarda la inscripción.
7. **Respuesta de MySQL.** Confirma la fila guardada o rechaza el email repetido gracias al índice único `(event_id, email)`.
8. **Respuesta de Laravel.** Si todo es correcto, redirige a la pantalla de confirmación. Si hay errores, vuelve al formulario con un mensaje por campo («Este email ya está inscrito en este evento», «No quedan plazas…»).
9. **Actualización.** Inertia.js pasa a React los nuevos datos (referencia o errores) sin recargar la página completa.
10. **Resultado en pantalla.** El asistente ve «Inscripción confirmada» con su referencia, o los errores junto a cada campo con lo que ya había escrito.

3. Tratamiento de datos

3.1 Entidades y relaciones



Un organizador tiene muchos eventos; un evento tiene muchos tipos de entrada y muchas inscripciones; cada inscripción corresponde a un tipo de entrada. Se simplifica Attendize: un pedido con un asistente pasa a ser una única «inscripción», y desaparecen cuentas, pedidos, pagos e impuestos.

3.2 Validaciones

Campo	Regla (se aplica siempre en Laravel; en React solo como ayuda)	Mensaje
`events.title`	Obligatorio, texto de 3 a 120 caracteres, sin espacios al inicio ni al final.	«El título debe tener entre 3 y 120 caracteres»
`events.venue`	Obligatorio, 1 a 150 caracteres.	«Indica el lugar del evento»
`events.starts_at / ends_at`	Fechas válidas; inicio no anterior a la fecha actual al crear; fin posterior al inicio.	«La fecha de fin debe ser posterior al inicio»
`events.capacity`	Entero entre 1 y 5.000; al editar, no menor que el número de inscritos.	«El aforo debe ser un número entero entre 1 y 5.000»
`events.description`	Opcional, máximo 5.000 caracteres; se guarda como texto plano (se elimina HTML).	«La descripción no puede superar 5.000 caracteres»
`ticket_types.name / capacity`	Nombre de 1 a 60 caracteres, único dentro del evento; plazas enteras ≥ 1; suma de plazas ≤ aforo del evento.	«La suma de plazas (X) supera el aforo (Y)»
`registrations.first_name / last_name`	Obligatorios, 1 a 80 caracteres.	«Introduce tu nombre» / «Introduce tus apellidos»
`registrations.email`	Obligatorio, formato email, máx. 190; se guarda en minúsculas y sin espacios; único por evento.	«Introduce un email válido...» / «Este email ya está inscrito en este evento»
`registrations.ticket_type_id`	Debe pertenecer al evento y tener plazas libres en el momento de guardar (comprobado dentro de la transacción).	«No quedan plazas para este tipo de entrada»

Aceptación de datos	Casilla obligatoria; no se almacena ningún otro dato personal (minimización).	«Debes aceptar el tratamiento de datos para inscribirte»

3.3 Correspondencia entre datos antiguos y nuevos

Tablas y campos según el esquema de Attendize v2.8.0 (migraciones de `database/migrations`). Se revisarán al implementar el script de migración (tarea T-16).

Attendize v2.8.0	Nuevo modelo	Acción	Regla
`users.first_name` + `last_name`	`users.name`	Transformar	Se unen con un espacio; si ambos están vacíos se usa la parte del email anterior a «@».
`users.email`	`users.email`	Conservar	Minúsculas y sin espacios. Si se repite, se conserva el usuario más antiguo.
`users.password`	`users.password`	Conservar	Ya está cifrada con bcrypt, compatible con Laravel; nunca se descifra.
`accounts.*`, `organisers.*`	—	Descartar	TechFest es un único organizador; el evento se asigna directamente al usuario (`events.user_id`).
`events.title`	`events.title`	Conservar	Se recorta a 120 caracteres; si queda con menos de 3, el evento pasa a borrador para revisión.
`events.description`	`events.description`	Transformar	Se elimina el HTML/Markdown y se guarda texto plano (máx. 5.000).
`events.venue_name` (+ campos `location_*`)	`events.venue`	Transformar	Se usa `venue_name`; si está vacío se compone con `location_address`; si sigue vacío: «Por confirmar» y borrador.
`events.start_date` / `end_date`	`starts_at` / `ends_at`	Conservar	Si falta el fin o es anterior al inicio, el evento pasa a borrador y se incluye en el informe.
`events.is_live`	`events.is_published`	Transformar	1 → publicado, 0 → borrador. Un evento sin tipos de entrada gratuitos se migra como borrador.
— (no existe)	`events.capacity`, `events.slug`	Calcular	Aforo = suma de plazas de sus tipos; slug = título normalizado + id para que sea único.
`events.currency_id`, `organiser_fee_*`, `social_*`, `barcode_type`, `bg_*` …	—	Descartar	Funciones fuera de alcance (pagos, redes sociales, diseño de entradas).
`tickets.title`	`ticket_types.name`	Conservar	Recorte a 60 caracteres; nombres repetidos en el mismo evento reciben sufijo « (2)».
`tickets.quantity_available`	`ticket_types.capacity`	Conservar	Si es nulo (ilimitado en Attendize) se usa el número de inscritos de ese tipo (sin plazas libres) y se marca para que el organizador fije las plazas.
`tickets.price`	—	Validar	Solo se migran tipos con precio 0. Los de pago y sus asistentes se listan en el informe para decisión del organizador.
`tickets.quantity_sold`	—	Recalcular	No se copia: las plazas ocupadas se calculan contando `registrations`.
`attendees.first_name / last_name`	`registrations.first_name / last_name`	Conservar	Si falta alguno se guarda «(sin dato)» y se incluye en el informe de revisión.

`attendees.email`	`registrations.email`	Validar	Minúsculas y sin espacios; si está vacío o no es un email válido, la fila **no se migra** y se anota en el informe.
`orders.order_reference` + `attendees.reference_index`	`registrations.reference`	Transformar	Se conserva la referencia original («REF-1») para que las confirmaciones ya enviadas sigan siendo válidas.
`attendees.has_arrived` / `arrival_time`	`registrations.checked_in_at`	Transformar	Si `has_arrived` = 1 se copia la hora; si no, nulo.
`attendees.is_cancelled` / `is_refunded`	—	Descartar	Las inscripciones canceladas o reembolsadas no se migran.
`orders.amount`, `payment_*`, `taxamt`, `gateways.*`	—	Descartar	Pagos fuera de alcance.

3.4 Reglas para valores incompletos o duplicados

1. **Copia previa obligatoria:** antes de migrar se hace `mysqldump` de la base de datos de Attendize y se comprueba que se puede restaurar.
2. **Duplicados de inscripción** (mismo evento y mismo email normalizado): se conserva la inscripción más antigua y las demás se escriben en `informe-duplicados.csv`.
3. **Datos obligatorios ausentes:** sin email → no se migra; sin nombre o apellidos → «(sin dato)» y revisión; evento sin lugar o fechas incoherentes → borrador.
4. **Idempotencia:** el script puede ejecutarse varias veces sin duplicar filas, porque comprueba si cada referencia ya existe.
5. **Verificación:** al terminar se compara número de eventos, tipos e inscripciones migrados + descartados con el total de origen. Si no cuadra, se deshace la transacción.